SOFTWARE ENGINEERING LAB – LAB 7

Fuzzing for Input Validation Bugs (Hypothesis)

Section H | Assigned repo: SETAPESU26/L6_L7_27_traffic_fine (Traffic Fine Calculator)
Student Name: Samvith Vaddiparthi | SRN: PES1UG24AM468 | Date: 12 November 2026

1. Objective

Use property-based fuzz testing with the Hypothesis library to find input-validation bugs in `Lab-7/traffic_fine.py`, fix the program so it handles any unexpected input safely, and confirm the fix by re-running the fuzz tests.

The module under test has three functions:

Function	Input	Purpose
`parse_ticket_id`	`"TF-2024-0456"`	Split a ticket ID into (prefix, year, serial)
`total_fines`	`"500,1200,300"`	Add up comma-separated fines, capped at 10000
`speed_excess`	`"72/60"`	Percentage by which the speed exceeds the limit

Environment: macOS, Python 3.12.4, pytest 9.2.0, Hypothesis 6.170.15 (installed with `pip install hypothesis pytest` in a virtual environment).

2. Step 2 – Running the buggy application

```
python traffic_fine.py
```

With the example inputs the program works. But a ticket ID without dashes (`TF2024`), an empty fine amount (`500,,300`) or a speed limit of zero (`72/0`) crashes the program with an unhandled exception.

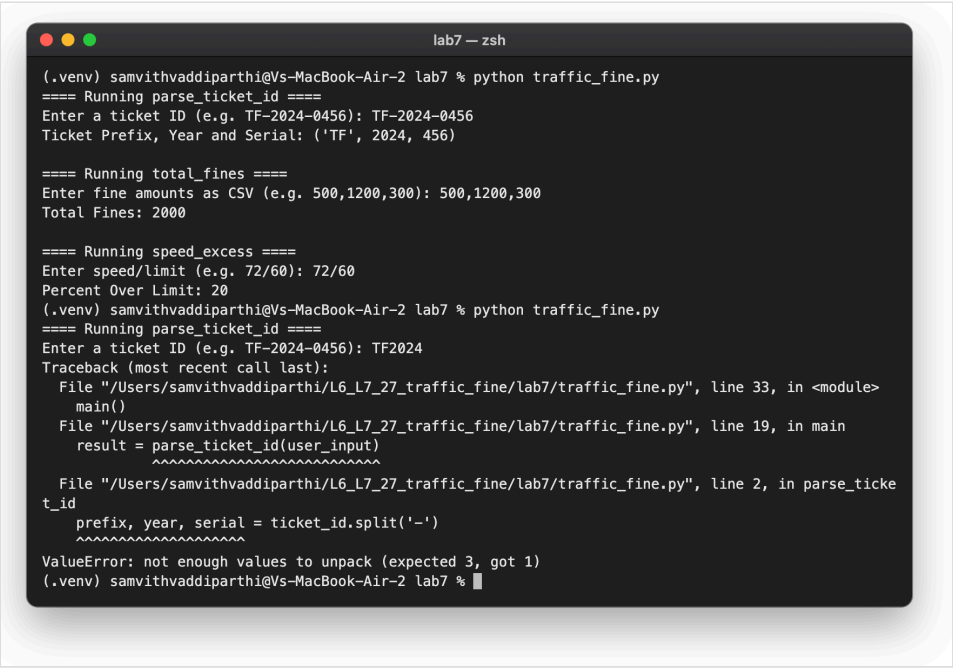


Figure 1 – Buggy program: valid input works; `TF2024` crashes parse_ticket_id with ValueError

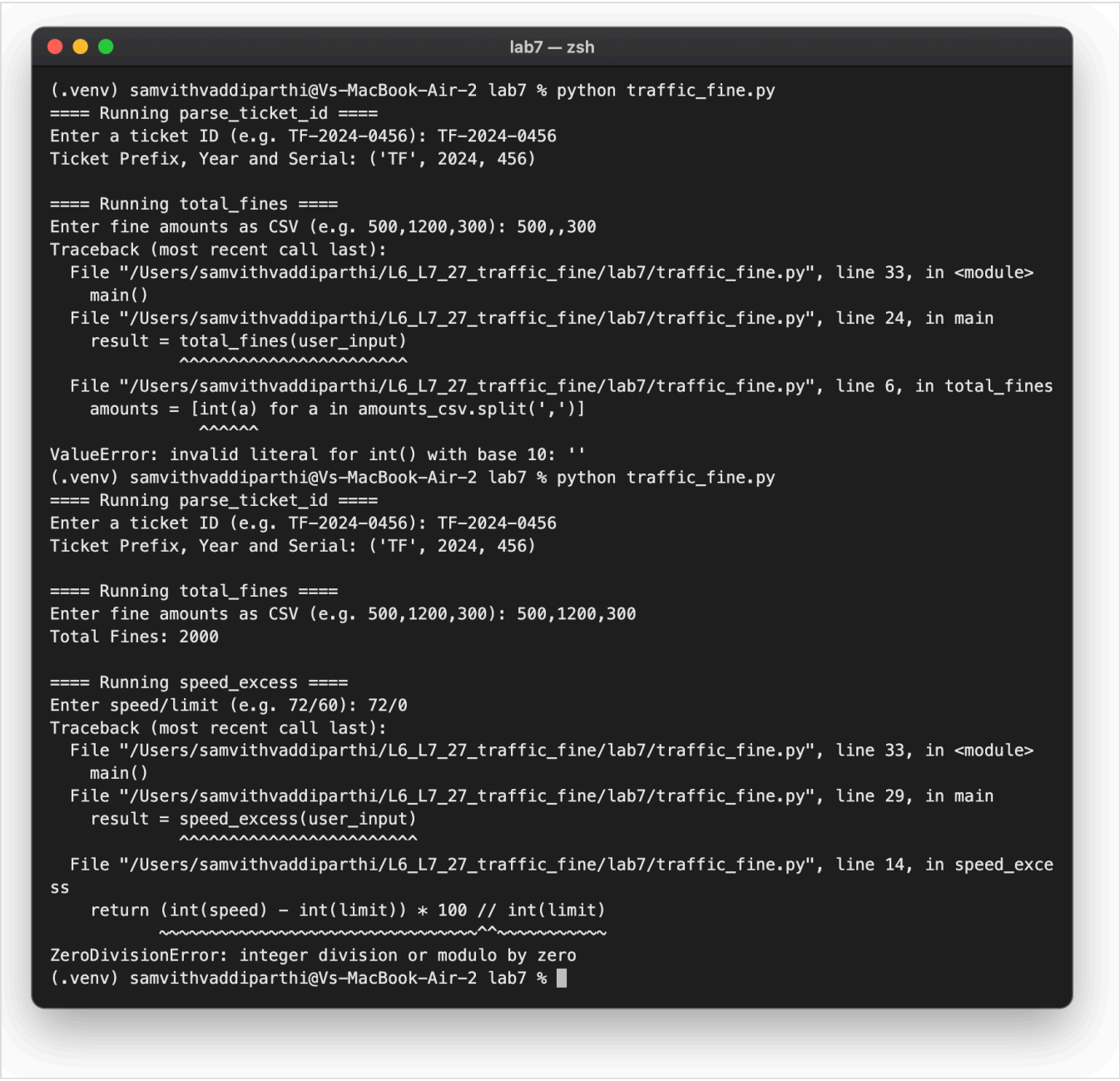


Figure 2 – Buggy program: `500,,300` crashes total_fines (ValueError); `72/0` crashes speed_excess (ZeroDivisionError)

3. Step 3 – Completing test_traffic_fine.py with Hypothesis

The starter file had three empty `@given(st.text() | st.none())` tests. I completed them and added property tests for each function. Purely random text rarely looks like `"TF-2024-0456"`, so I also built format-shaped strategies: random pieces joined with `-`, `,` or `/`, and integer pairs. Every test runs 500 generated examples (`settings(max_examples=500)`).

Test	Strategy (generated input)	Property checked
`test_parse_ticket_id_no_crash`	`st.text() | st.none()`	Never raises; returns a 3-tuple or None
`test_parse_ticket_id_malformed_no_crash`	0–5 random pieces joined with `"-"`	Never raises; any accepted result has an upper-case letter prefix, a 4-digit year and a serial ≥ 0
`test_parse_ticket_id_valid`	letters + 4-digit year + digit serial	Returns `(PREFIX, year, int(serial))`
`test_total_fines_no_crash`	`st.text() | st.none()`	Never raises; result is None or 0 ≤ total ≤ 10000
`test_total_fines_malformed_in_range`	1–6 random pieces joined with `","`	Never raises; result is None or in [0, 10000]
`test_total_fines_never_negative`	lists of integers (incl. negatives) as CSV	Total never negative; a negative fine is rejected
`test_total_fines_valid`	lists of non-negative integers	Equals `min(sum, 10000)`
`test_speed_excess_no_crash`	`st.text() | st.none()`	Never raises; result is None or ≥ 0
`test_speed_excess_malformed_numbers`	two integers in [-500, 500] as `"s/l"`	Never raises; percentage never negative
`test_speed_excess_valid`	speed 0–400, limit 1–200	Correct percentage, or 0 when not over the limit

The `no_crash` tests follow the pattern from the handout:

```
@given(st.text() | st.none())
def test_parse_ticket_id_no_crash(s):
    try:
        result = parse_ticket_id(s)
    except Exception as e:
        assert False, f"parse_ticket_id crashed on {s!r} with {type(e).__name__}: {e}"
    assert result is None or (isinstance(result, tuple) and len(result) == 3)
```

4. Step 4 – Running the tests on the buggy code

```
pytest test_traffic_fine.py
```

Result: 8 failed, 2 passed. Only the two "valid input" tests for parse_ticket_id and total_fines passed. Hypothesis shrank each failure to a minimal input. These are the bugs it found:

#	Function	Minimal input found	Behaviour of the original code
1	parse_ticket_id	`None`	**AttributeError: 'NoneType' object has no attribute 'split'**
2	parse_ticket_id	`''`	**ValueError: not enough values to unpack (expected 3, got 1)**
3	parse_ticket_id	`'--'`	**ValueError: invalid literal for int() with base 10: ''**
4	parse_ticket_id	`'-0- 7 '`	**Silently accepted: returns ('', 0, 7) with an empty prefix and year 0**
5	total_fines	`None`	**AttributeError: 'NoneType' object has no attribute 'split'**
6	total_fines	`''`	**ValueError: invalid literal for int() with base 10: ''**
7	total_fines	`'-1'`	**Returns -1 (a negative total of fines)**
8	total_fines	`'1,-1'`	**Returns 0 (a negative "fine" cancels a real one)**
9	speed_excess	`None`	**AttributeError: 'NoneType' object has no attribute 'split'**
10	speed_excess	`''`	**ValueError: not enough values to unpack (expected 2, got 1)**
11	speed_excess	`'/'`	**ValueError: invalid literal for int() with base 10: ''**
12	speed_excess	`'0/0'`	**ZeroDivisionError: integer division or modulo by zero**
13	speed_excess	`'0/1'`	**Returns -100 (negative "percent over limit")**

Bugs 4, 7, 8 and 13 don't crash. The program runs and returns a wrong answer. A plain "no crash" test would miss them; the range and validity properties caught them. Also, `int()` silently accepts inputs such as `'+5'`, `' 7 '`, `'1_000'` and non-ASCII digits like `'٣'`, which a ticket or fine field should not allow.

The full pytest output does not fit on one screen, so it is shown below in 7 consecutive screenshots (Figures 3–9).



Figure 3 – Failing tests (1/7): parse_ticket_id_no_crash – '' and None



Figure 4 – Failing tests (2/7): parse_ticket_id_malformed_no_crash – '-0- 7 ', '--', ''

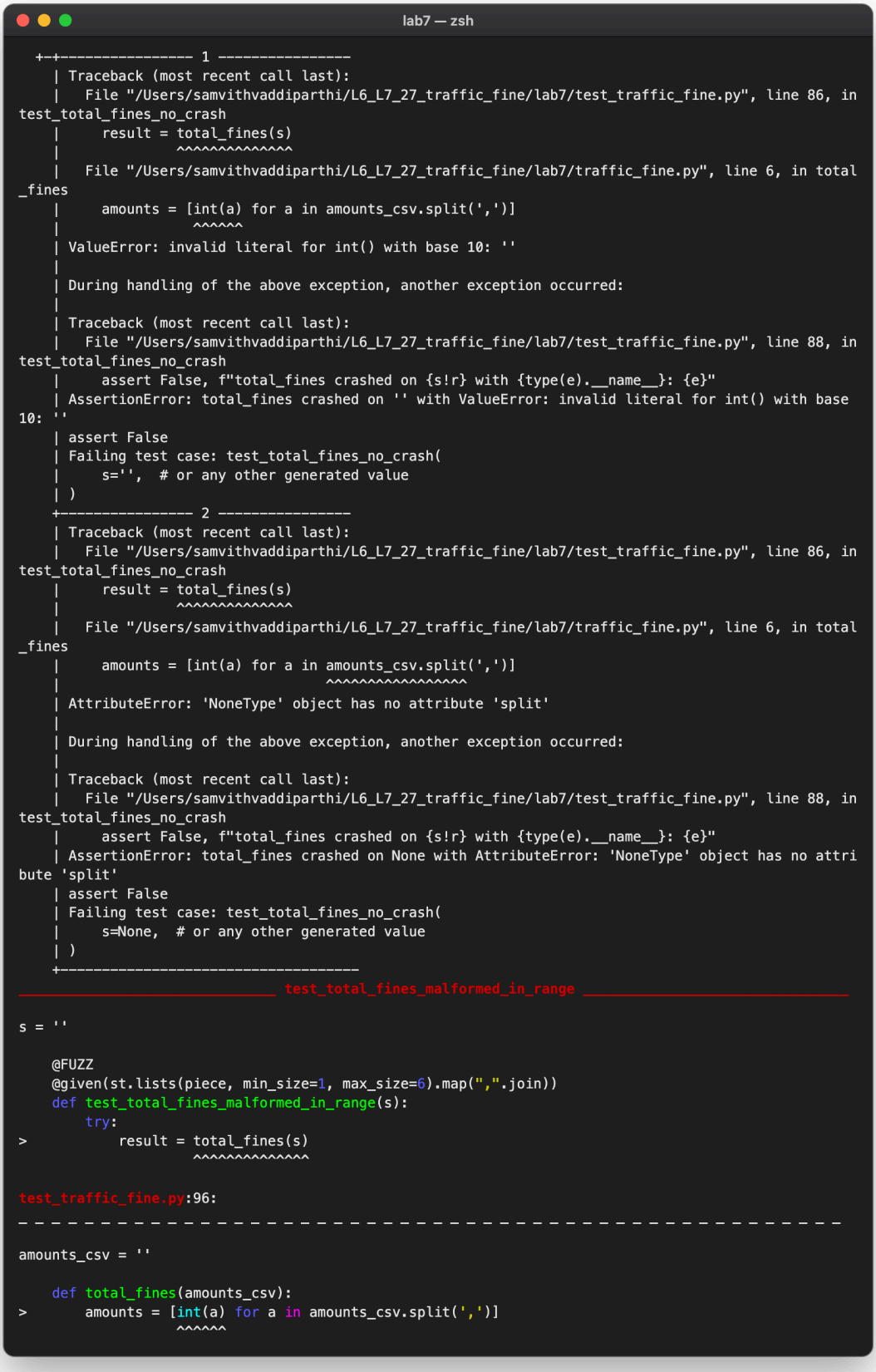


Figure 5 – Failing tests (3/7): total_fines_no_crash – '' and None



Figure 6 – Failing tests (4/7): total_fines_malformed_in_range ('') and total_fines_never_negative ('1,-1', '-1')



Figure 7 – Failing tests (5/7): speed_excess_no_crash ('/' and '')

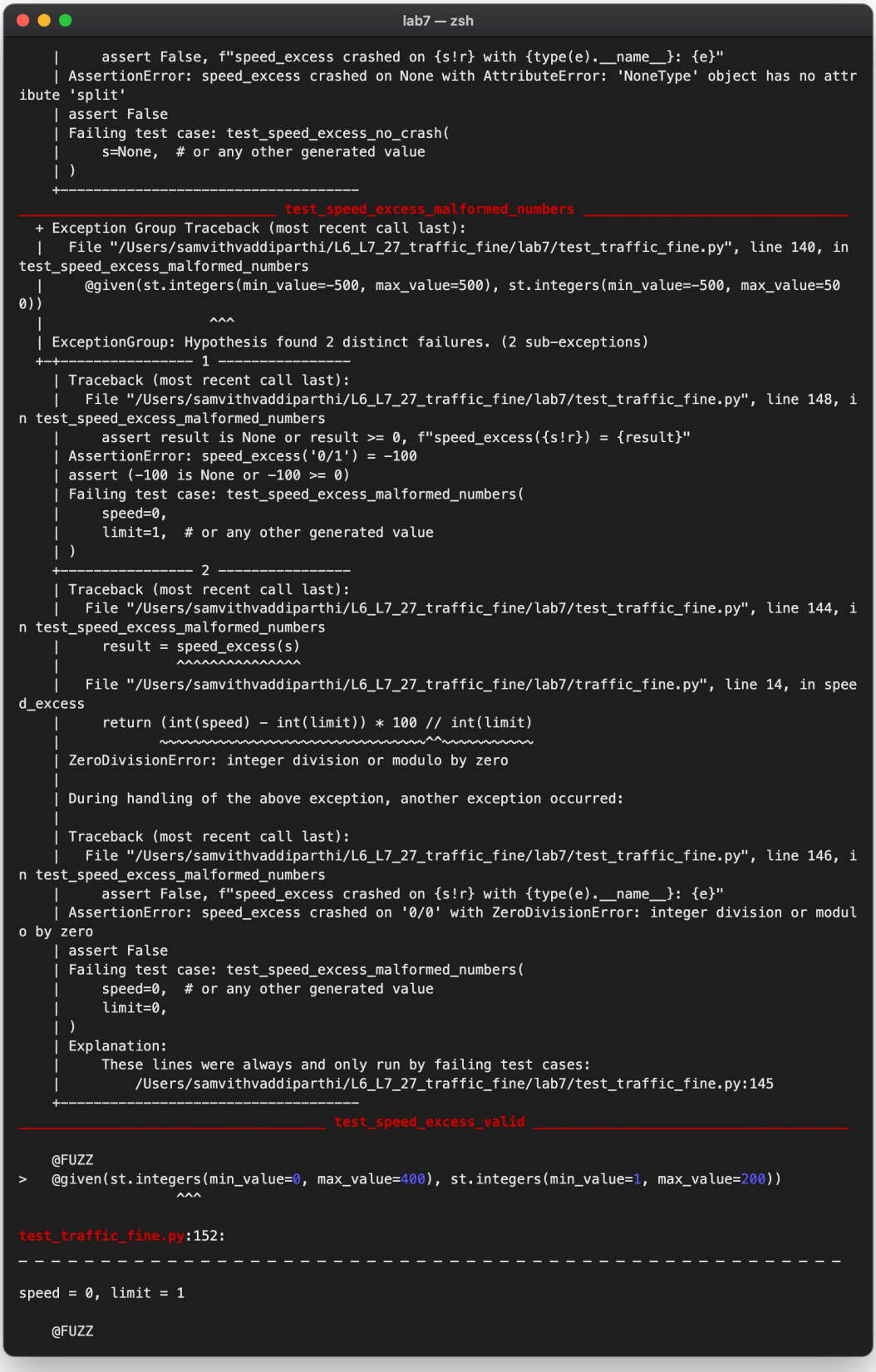


Figure 8 – Failing tests (6/7): speed_excess_no_crash (None) and speed_excess_malformed_numbers ('0/1', '0/0')

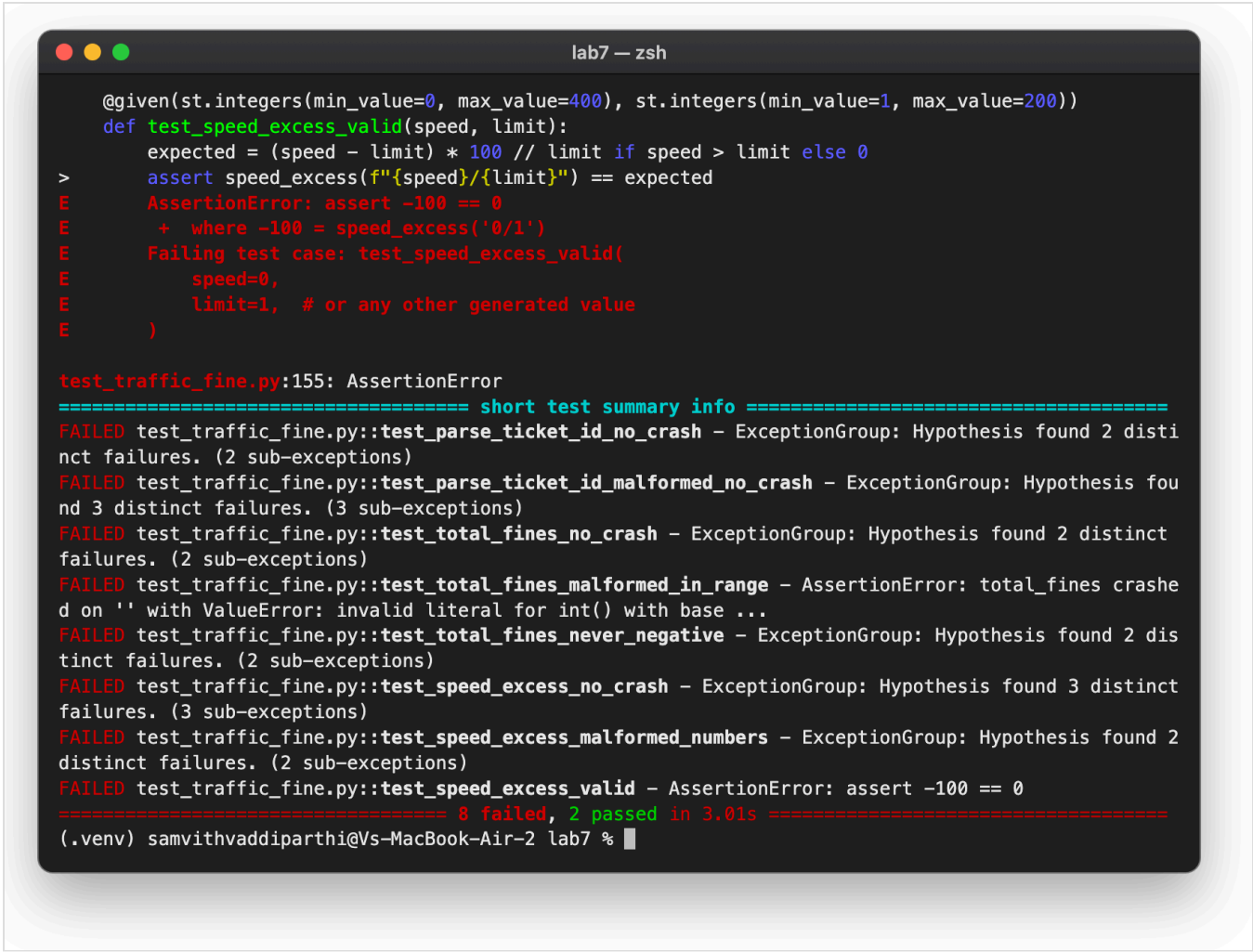


Figure 9 – Failing tests (7/7): speed_excess_valid and the summary – 8 failed, 2 passed

5. Step 5 – Fixing traffic_fine.py

All three functions now follow one rule: **invalid input returns `None` and never raises**. `main()` checks for `None` and prints a helpful message instead of a traceback.

Problem (bug #)	Fix
None / non-string input (1, 5, 9)	`isinstance(value, str)` check at the start of every function
Wrong number of parts (2, 10)	`len(parts)` is checked before unpacking: 3 parts for a ticket, 2 for a speed reading
Empty / non-numeric numbers (3, 6, 11), and `'+5'`, `'1_000'`, non-ASCII digits	New helper `_to_int(text, max_digits)`: trims spaces, accepts only plain ASCII digits within a digit limit, otherwise returns None
Empty prefix / bad year accepted (4)	Prefix must be 1–5 ASCII letters; year must be exactly 4 digits
Negative fines (7, 8)	`_to_int` rejects the minus sign, so negative amounts make the whole input invalid
Division by zero (12)	A speed limit of 0 is rejected
Negative percentage (13)	Returns 0 when `speed <= limit` (the driver was not over the limit)

```
def _to_int(text, max_digits):
    if not isinstance(text, str):
        return None
    text = text.strip()
    if not text or len(text) > max_digits:
        return None
    if not (text.isascii() and text.isdigit()):
        return None
    return int(text)

def speed_excess(reading):
    if not isinstance(reading, str):
        return None
    parts = reading.split('/')
    if len(parts) != 2:
        return None
    speed = _to_int(parts[0], max_digits=4)
    limit = _to_int(parts[1], max_digits=4)
    if speed is None or limit is None or limit == 0:
        return None
    if speed <= limit:
        return 0
    return (speed - limit) * 100 // limit
```

Valid inputs give exactly the same results as before (for example `TF-2024-0456 → ('TF', 2024, 456)`, `500,1200,300 → 2000`, `72/60 → 20`).

```
python traffic_fine.py
```

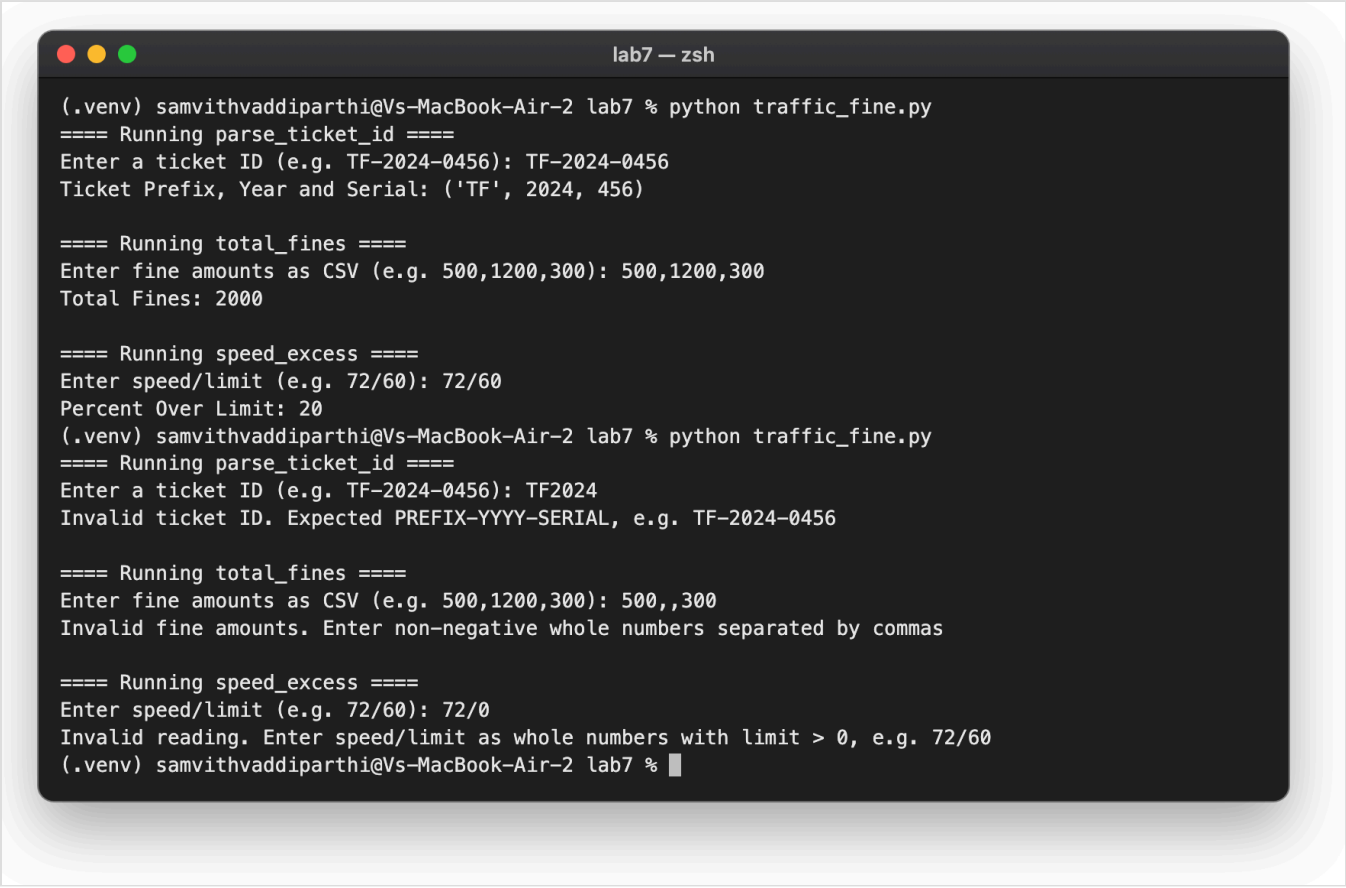


Figure 10 – Fixed program: valid input gives the same output as before; the inputs that crashed it before (TF2024, 500,,300, 72/0) now show a clear message

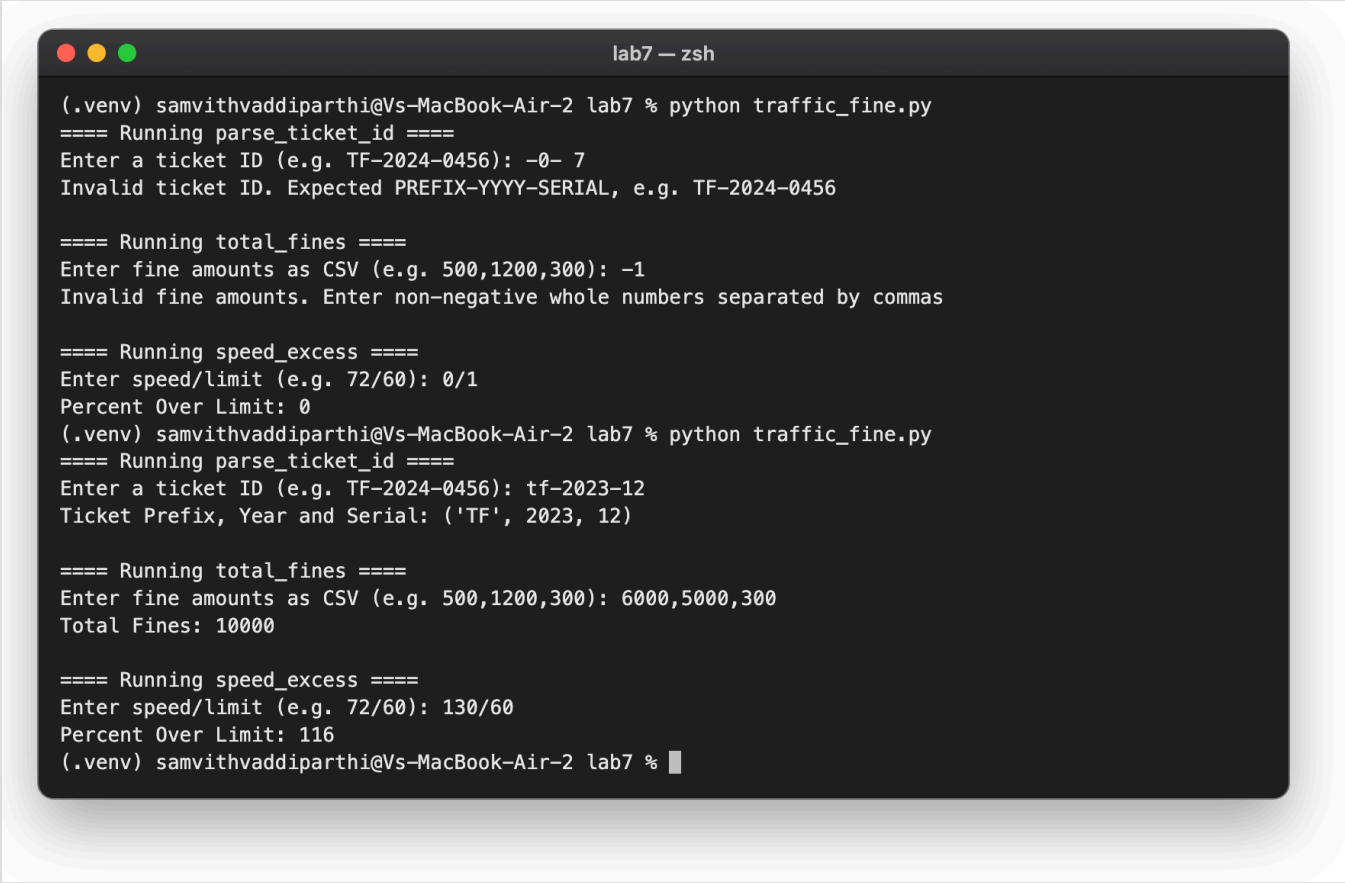


Figure 11 – Fixed program: fuzzer-found inputs ('-0- 7 ', '-1', '0/1') are handled correctly; lower-case prefix, the 10000 cap and 130/60 still work

6. Step 6 – Re-running the tests on the fixed code

```
pytest test_traffic_fine.py
pytest test_traffic_fine.py -v
```

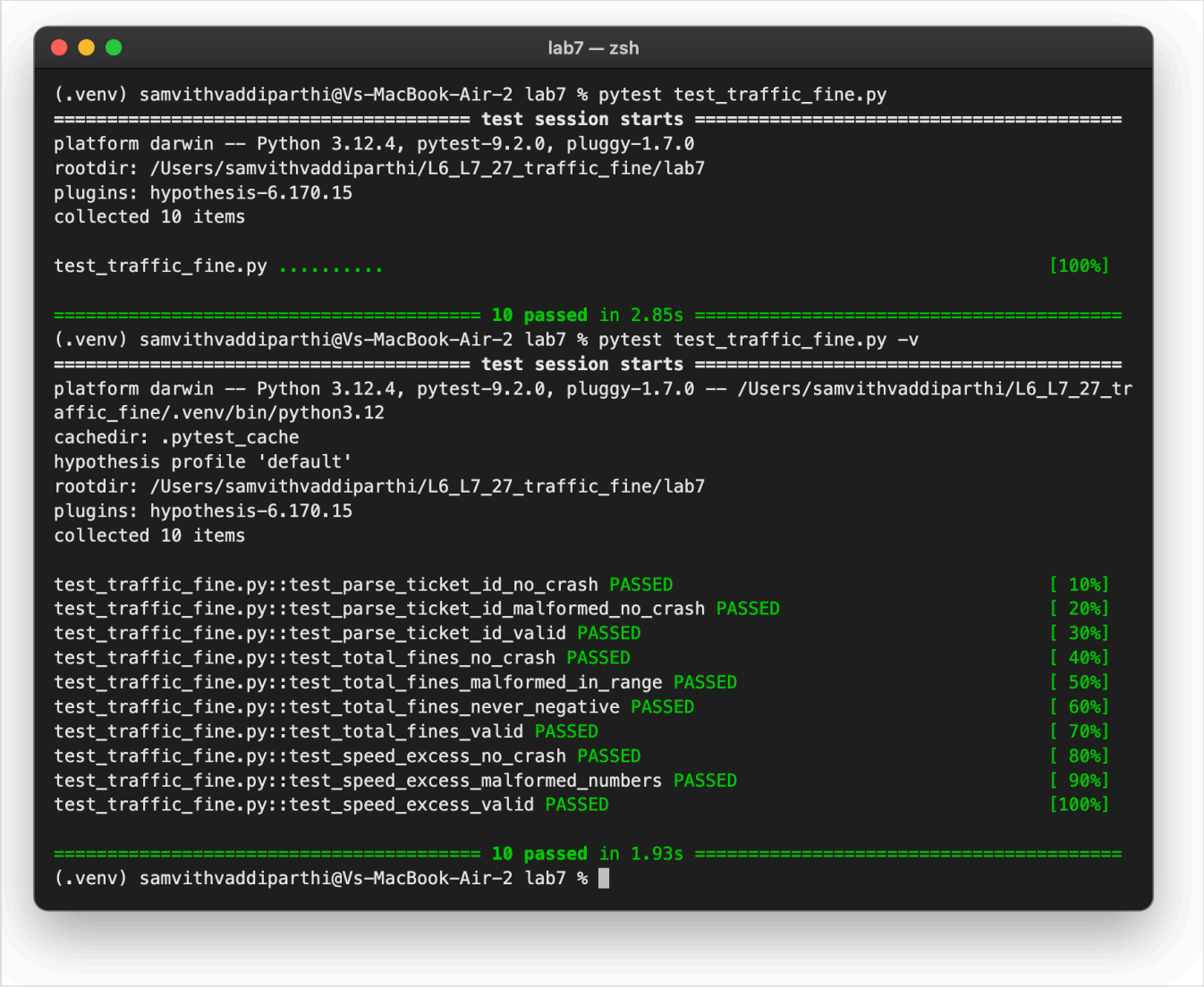


Figure 12 – All 10 Hypothesis tests pass on the fixed code (500 generated inputs per test)

Result: 10 passed, 0 failed. As an extra check, I ran a separate stress test of 20,000 random strings, None values and Latin-1 bytes through all three functions. It found no crashes.

7. Conclusion

- Hypothesis found **13 distinct input-validation bugs** across the three functions. Each was shrunk to a minimal input such as `None`, `''`, `'0/0'` or `'-1'`.
- Four of the bugs did not crash; they returned wrong answers. Only property checks (range and validity) caught these, not crash checks.
- Format-shaped strategies (pieces joined with the real separators) found bugs that pure random text rarely reaches, such as `'-0- 7 '` and `'0/0'`.
- After defensive validation the program never crashes, rejects bad data clearly, and gives the same results for valid input.

Deliverables (GitHub): `Lab-7/traffic_fine.py` (fixed script), `Lab-7/test_traffic_fine.py` (Hypothesis tests), `Lab-7/screenshots/` and this PDF.